CAN-Driven Digital Instrument Cluster on STM32F469

Sana Gholipour Bashiri

Abstract

This project implements a CAN-driven digital instrument-cluster interface on an STM32F469I-DISCO platform using TouchGFX and an RTOS-based application. CAN2 receives speed and time messages through identifier-list filters, and a periodic GUI callback updates a speed gauge and a 24-hour digital clock. The saved configuration specifies a 125 kbit/s CAN bit rate and a 480 x 800 pixel display. The implementation separates peripheral initialization, a polling receive task, and display rendering, while sharing the decoded values through global variables. Source-based analysis confirms the principal data path and identifies limitations in bit-field masking, fractional speed display, error handling, and inter-task consistency. The report presents the implementation and analytical results without claiming hardware measurements that are not available in the repository.

1. Introduction

An embedded instrument display converts bus messages into information that a user can interpret immediately. Reliable behavior requires correct reception, decoding, unit conversion, and rendering. Each stage has different responsibilities: the CAN controller handles frames, application code interprets payload fields, and the graphical interface presents the resulting values. Errors at one stage can remain visually hidden when only nominal inputs are used.

The STM32F469 Discovery platform provides a microcontroller, display interfaces and memory resources suited to graphical applications. Its board documentation describes the display and expansion facilities [5]. In this project, STM32 HAL code configures the peripherals, CMSIS-RTOS2 creates the application threads, and TouchGFX supplies the display framework. The implemented interface contains a speed gauge and digital clock; the supplied source is a TouchGFX implementation.

1.1 Implementation scope

The examined artifacts include Core/Src/main.c, the STM32CubeMX IOC file, TouchGFX Designer content, generated GUI code, board drivers, and IDE project definitions. There is no supplied LVGL implementation, as-built external transceiver schematic, hardware demonstration video, or instrumented trace for this project. Accordingly, the results in this report are derived from the implementation and calculated payload examples. They establish what the code does and what remains to be verified experimentally, rather than a completed characterization of a physical prototype.

2. Hardware configuration and software architecture

The saved IOC configuration identifies an STM32F469 platform with a 180 MHz system clock and 45 MHz APB1 peripheral clock. CAN2 operates in normal mode. PB13 is configured as CAN2_TX and PB12 as CAN2_RX with alternate function AF9. Both CAN1 and CAN2 peripheral clocks are enabled by the HAL support code. An external CAN physical-layer transceiver is required between these controller pins and the bus; the exact component and its support circuit are not supplied.

Table 1. Configuration recovered from the IOC and source files.

Item	Saved setting
Target	STM32F469I-DISCO
System / CAN peripheral clock	180 MHz / 45 MHz
CAN2 timing	Prescaler 24; BS1 12 TQ; BS2 2 TQ; SJW 1 TQ
CAN mode / bit rate	Normal / 125,000 bit/s
Receive task	defaultTask; normal priority; 512-byte stack
GUI task	TouchGFXTask; normal priority; 16,384-byte stack
Display / GUI version	480 x 800 pixels, 16 bpp / TouchGFX 4.25.0

2.1 CAN timing calculation

The bit-time configuration contains one synchronization quantum, twelve quanta in segment 1, and two in segment 2. The total is fifteen quanta per bit. Dividing 45 MHz by the prescaler of 24 and by fifteen yields 125,000 bit/s. The nominal sample point is (1 + 12)/15, or 86.7% of the bit interval. These are calculated configuration properties; they are not oscilloscope measurements of the external bus.

2.2 Runtime decomposition

Initialization configures GPIO, graphics-related peripherals, external memory interfaces, CAN2, and TouchGFX before the scheduler starts. The default task polls FIFO0 and writes shared clock and speed variables. The TouchGFX task runs the rendering framework, which calls the screen tick handler. That handler reads the variables and updates the widgets. The Model class has an empty tick function, so the current implementation bypasses a populated model-to-presenter data path [1-4].

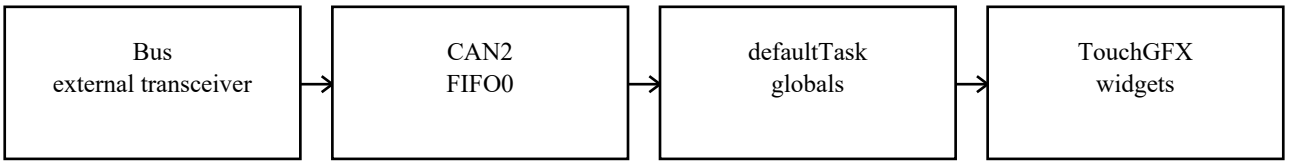


Figure 1 summarizes the implemented data path. The transceiver is an external dependency; the remaining stages are represented by source in simulation/.

3. Message reception and decoding

The application uses two 32-bit identifier-list filter banks assigned to FIFO0. Bank 7 lists standard identifiers 0x101 and 0x220. Bank 8 lists 0x430 and a second zero-valued entry. The CAN2 filter split is configured at bank 7. Because the second entry in bank 8 is zero, standard identifier 0x000 can also match that list; the receive loop has no processing branch for it. Filtering is therefore slightly broader than the three application identifiers alone.

Table 2. Payload handling implemented in StartDefaultTask.

Identifier	Received application data	Current handling
0x101	VEH_SPD: 12-bit speed	Combine bytes 1 and 0 into uint16_t
0x430	RTC seconds / minutes / hours	Copy bytes 0 / 1 / 2
0x220	Fuel state in bits 6-7 of byte 0	Accepted by filter; no decode branch
0x000	No defined application field	May pass filter; no decode branch

The receive loop calls HAL_CAN_GetRxMessage once per iteration. A successful reception toggles LED4 and dispatches according to StdId. For 0x430, Sec, Min, and Hour are copied directly from the first three bytes. For 0x101, Vehicle_speed is assigned the little-endian two-byte value. The task then delays for ten RTOS ticks. This is a polling implementation; enabling the CAN receive interrupt in the support code does not by itself make payload processing interrupt driven.

```
Vehicle_speed = ((uint16_t)RxData[1] << 8) | RxData[0];
```

The intended speed field is twelve bits wide. A decoder aligned with that layout would mask the combined value with 0x0FFF before conversion. The supplied expression does not apply that mask, so unrelated upper bits in byte 1 can contaminate the result. The hour field similarly occupies six bits, but the code copies the entire byte. These differences are immaterial only while the unused payload bits remain zero.

3.1 Reception integrity

The current code checks the HAL return status but does not validate DLC, IDE, RTR, or clock ranges before using payload bytes. It does not explicitly handle the speed fault code 0xFFE. Failed receptions lead to no recovery action, and Error_Handler is empty. These observations define specific hardening work: validate frame shape, mask signal fields, distinguish invalid signals from zero, and propagate bus or display faults to an observable status.

4. Graphical interface and analytical results

The generated screen places a 24-hour clock at x = 112, y = 144 with a 257 x 90 pixel area. A 388 x 388 pixel gauge begins at x = 46, y = 329. Its range is 0-200, with a needle sweep from -110 to 112 degrees. The generated initial values are 10:10:00 for the clock and 200 for the gauge. These are design-time defaults, not evidence of received CAN information [3].

On each screen tick, the callback calculates final_speed = Vehicle_speed x 0.125 and stores the result in a uint16_t. The clock is refreshed when any field differs from its backup value, and the display is invalidated after setTime24Hour. The gauge is updated using setValue. Table 3 shows deterministic examples of the current conversion. All cases are calculated from the source, with unused bits cleared unless stated otherwise.

Table 3. Calculated decoding cases; no hardware measurement is implied.

Input bytes / frame	Current displayed value	Interpretation
20 03 / 0x101	100 km/h	Raw 800 x 0.125
AB 01 / 0x101	53 km/h	53.375 truncated to integer
C0 03 / 0x101	120 km/h	Exact integer point
20 F3 / 0x101	7780 before gauge handling	Upper four bits are not masked
28 23 0E / 0x430	14:35:40	Direct byte extraction

4.1 Update behavior

The clock backup values are stored in the same units as the received fields. The gauge backup logic differs: the condition compares raw Vehicle_speed with gauge1Bkp, but after an update the backup receives the scaled integer final_speed. For a raw value of 800, the next comparison is therefore 800 versus 100. The callback can repeatedly update an unchanged gauge. Comparing a scaled value with a scaled backup, or retaining the raw value consistently, would avoid this redundant work.

A second boundary case follows from initialization. With zero-initialized global variables and zero-initialized backups, the first callback may not overwrite the generated defaults when no message has arrived. The screen can therefore initially show 10:10:00 and a 200-valued gauge until changes trigger updates. A dedicated data-valid flag would provide a clearer startup state.

The accompanying results directory contains a source-derived architecture figure and calculated test vectors. No fabricated board photograph, measured refresh rate, or synthetic hardware success claim is included. The visual configuration and calculation results should be used to plan, rather than replace, a bench validation.

5. Reliability and validation considerations

The receive task and GUI callback communicate through shared global variables. There is no queue, mutex, timestamped snapshot, or explicit data-valid flag in this path. Even if individual accesses happen to be indivisible on the target, a three-field clock update is not guaranteed to be observed as a coherent transaction. The screen can read one field before a receive update and another afterward. A short protected snapshot or message queue would make the transfer explicit and preserve coherent time values.

5.1 Capacity and timing

Each successful receive-loop iteration removes at most one frame and then waits ten RTOS ticks. The actual service interval includes task scheduling and display workload in addition to that delay. The code does not drain all pending FIFO entries before yielding, and it does not expose FIFO overrun statistics. Thus, the source alone does not establish the maximum sustainable traffic load or an end-to-end latency bound. These need timestamped bus and firmware measurements.

At the nominal message rates of 100 ms for speed and fuel and 1000 ms for time, application traffic is modest, but a hardware test should include background traffic and bursts. Observing only the displayed speed would not reveal every dropped frame. A reception counter, overflow counter, and age-of-last-update indicator would make such a test interpretable. The empty error handler should also be replaced before treating initialization failures as diagnosed conditions.

5.2 Proposed acceptance procedure

Table 4. Bench tests to perform; these are proposed checks, not completed results.

Test	Acceptance observation
Known speed payloads	Compare 0, 100, 120, and 200 km/h with decoded values
Fractional speed	Confirm chosen rounding policy using raw 427
Clock boundaries	Check 23:59:59, rollover, and invalid byte values
Unused bits / reserved code	Mask upper speed/hour bits; identify 0xFFE as invalid
Missing frames / bursts	Expose stale-data indication and monitor overflow counters
Restart with silent bus	Show a defined no-data state rather than design defaults

The repository preserves the implementation rather than introducing untested functional changes. Its limitations are documented so that subsequent revisions can be reviewed against the same payload vectors. A consistent unsigned signal definition, fault handling policy, and unit convention should be agreed before extending the gauge beyond its 0-200 range or adding the currently unimplemented fuel display.

6. Reproduction and deliverables

The repository is self-contained with respect to authored firmware, board-support source, GUI definitions, image and text assets, generated GUI files, and IDE project descriptions. It deliberately excludes build products and the large TouchGFX platform SDK. The SDK is a tool dependency to restore from a matching STMicroelectronics installation, not a runtime result. Original third-party notices are retained; the package does not assign a new license to the code or assets.

The IOC file records STM32CubeMX 6.15.0, STM32Cube FW_F4 V1.28.2 and TouchGFX 4.25.0. Install those components or a verified compatible environment, restore the TouchGFX SDK directories, and open TouchGFX/can_touch.touchgfx to generate code. Import the STM32CubeIDE project or use the provided MDK-ARM, EWARM, or GCC project with its corresponding toolchain. Inspect the user-code sections when regenerating peripheral initialization. Build and flash only after resolving the documented dependencies.

For hardware evaluation, connect a suitable external transceiver to PB13/TX and PB12/RX and configure the test interface for 125 kbit/s. Confirm wiring, bus electrical settings and frame formats against the actual hardware documentation. Inject the calculated cases in results/decoding_vectors.csv and record observed values separately. No as-built electrical circuit is inferred from software pin assignments. The desktop simulator sources require a host data provider because the live globals are defined by the firmware main program.

7. Conclusion

The implementation establishes a clear embedded path from CAN2 reception to a TouchGFX speed gauge and clock. Its saved timing parameters agree analytically with 125 kbit/s, and nominal zero-padded payloads produce the expected integer speed and time values. The principal next steps are to make decoding conform exactly to the bit layout, enforce frame validity, transfer coherent data between tasks, and define startup and stale-data behavior. These findings support focused development while keeping the evidence boundary between code inspection and measured hardware performance explicit.

References

[1] Peripheral configuration and receive task: simulation/Core/Src/main.c and simulation/Core/Src/stm32f4xx_hal_msp.c.

[2] Saved platform settings: simulation/STM32F469I-DISCO.ioc.

[3] Screen behavior: simulation/TouchGFX/gui/src/screen1_screen/Screen1View.cpp and generated/gui_generated/src/screen1_screen/Screen1ViewBase.cpp.

[4] GUI project and model: simulation/TouchGFX/can_touch.touchgfx and simulation/TouchGFX/gui/src/model/Model.cpp.

[5] STMicroelectronics, UM1932, Discovery kit with STM32F469NI MCU, Rev. 5, March 2023. https://www.st.com/resource/en/user_manual/um1932-discovery-kit-with-stm32f469ni-mcu-stmicroelectronics.pdf. Accessed 24 October 2026.